

巨大コードの読解と性能改善で学ぶ 用 語まとめ(第 4 章)

～ 大規模コードの歩き方から、インシデント対応・N+1 問題の解決まで

～

この文書は、巨大なコードベースの理解と、注文系の性能トラブル対応でよく出てくる言葉を、身近な例えとセットで分かるようにまとめたものです。

作成日:2026 年 6 月 17 日

はじめに — この文書の使い方

この文書は、用語の意味を初学者向けにやさしくまとめたものです。むずかしい言葉は、レストランや倉庫などの身近なものにたとえて説明しています。

各用語は「ひとことで言うと → 身近な例え → 具体例 → つながり」の同じ型で並べています。まずは「ひとことで言うと」だけ拾い読みしても大丈夫です。

◆ この文書で扱う用語(テーマ別)

- **巨大なコードを読み解く:** Wiki / IntelliJ / grep / エントリーポイント / コンテキスト
- **設計の型と部品:** レイヤードアーキテクチャ / インターフェース / DI コンテナ経由 / ストラテジーパターン / ドメインモデル / ユーティリティ
- **コードの中の実物クラス:** SecurityConfig / JwtTokenProvider / UserDetailsServiceImpl / InventoryAllocationService / PricingEngine / テストスイート
- **トラブル対応の現場:** インシデント / アサイン / タイムアウト / スタックトレース / APM ツール / デバッグ
- **原因を突き止める(性能問題):** ブレークポイント / スロークエリログ / SQL / `OrderService.getOrderList()` / `OrderRepository.findAll()`
- **N+1 問題と解決策:** `OrderItemRepository.findById()` / `@EntityGraph` / JOIN FETCH / ページネーション / `InventoryService.getStockList()`

第 1 部 — 巨大なコードを読み解く

1. Wiki(ウィキ)

ひとことと言うと:チームの決まりや仕組みを、みんなで書き溜めて共有する社内の知識ノートのこと。

お店のバックヤードに貼ってある「業務マニュアル集」のようなものです。新しく入った人が、誰かに毎回聞かなくても、まずここを読めば全体像がつかめます。

◆ Wiki に書かれていること

- システム全体の構成や用語の説明
- 開発の手順やチームのルール
- 過去のトラブルとその対応の記録

【つながり】 Wiki で全体像をつかんだら、実際のコードを開いて読む道具が、次の「IntelliJ」です。

2. IntelliJ(インテリジェイ / IntelliJ IDEA)

ひとことと言うと:コードを書いたり読んだり探したりを快適にしてくれる、開発専用の高性能エディタのこと。

何でも揃ったプロ向けの作業デスクのようなものです。文房具(編集)も辞書(検索)も虫眼鏡(定義へジャンプ)も一体になっていて、巨大なコードでも迷子になりにくくなります。

◆ 大きなコードで助かる機能

- クラスやメソッドの定義へ一発でジャンプできる
- 「どこから呼ばれているか」を一覧できる
- プロジェクト全体を高速に検索できる

【つながり】 IntelliJ で読み始めるとき、まず探す「アプリの始まる場所」が、次の「エントリーポイント」です。

3. grep(グレップ)

ひとことと言うと:大量のファイルの中から、指定した文字を含む行を一瞬で探し出すコマンドのこと。

分厚い辞書で、目的の単語を一気に引き当てる索引のようなものです。800 ものファイルを1枚ずつめくる代わりに、キーワードのある場所だけを瞬時に教えてくれます。

◆ 使い方(イメージ)

```
# 「OrderService」という文字を含む行を全部探す
grep -r "OrderService" ./src
# -r = フォルダの中を丸ごとたどって探す
```

【つながり】 grep で見つけた場所が、コード全体のどの「位置・状況」なのかを示すのが、次の「コンテキスト」です。

4. エントリーポイント(Entry Point)

ひとことと言うと:プログラムが動き始める、いちばん最初の入り口にあたる場所のこと。

ビルの正面玄関のようなものです。中がどれだけ広く複雑でも、人はまずここから入ります。コードを読むときも、この入口から順にたどると流れが分かります。

◆ エントリーポイントの例

```
// Java アプリが最初に動き出す場所
public static void main(String[] args) {
    ...ここからプログラムが始まる...
}
```

【つながり】 入口から処理を追ううちに迷わないために意識する「今どこの話か」が、次の「コンテキスト」です。

5. コンテキスト(Context)

ひとことと言うと:いま見ているコードが「どんな状況・前提のもとで動いているか」という背景情報のこと。

会話の「いまは何の話をしているか」という前後の流れのようなものです。同じ一言でも場面が違えば意味が変わるように、コードも前後の状況を踏まえないと正しく読めません。

◆ コンテキストに含まれるもの

- 今ログインしているのは誰か(利用者の情報)
- どの画面・どの処理から呼ばれたか
- 今どんな設定で動いているか

【つながり】 こうした全体の前提を整える土台となる「層に分けた設計の考え方」が、次の「レイヤードアーキテクチャ」です。

第2部 — 設計の型と部品

6. レイヤーアーキテクチャ(Layered Architecture)

ひとことで言うと:役割ごとにコードを層に分けて積み重ねる、整理された設計の考え方のこと。

レストランを「客席(受付)・厨房(調理)・倉庫(保管)」と役割で分けるようなものです。各層は自分の仕事だけに集中し、隣の層へ順番に渡すので、全体が見通しよくなります。

◆ よくある3つの層

層	役割	たとえ
Controller 層	外からの要求を受け取る	注文を受ける客席
Service 層	中身の処理・判断をする	調理する厨房
Repository 層	データを出し入れする	保管する倉庫

【つなぎり】 層と層をきれいに繋ぐために、各層が約束する「窓口の形」が、次の「インターフェース」です。

7. インターフェース(Interface)

ひとことで言うと:「どんな操作ができるか」だけを先に決めておく、中身を伴わない約束ごとのこと。

家電のリモコンのボタン配置のようなものです。「電源」「音量」といったボタン(できること)だけ決めておけば、中の仕組みが違うテレビでも同じように扱えます。

◆ 書き方(イメージ)

```
// 「できること」だけを宣言する(中身は書かない)
interface PricingEngine {
```

```
int calculatePrice(Order order);  
}
```

【つながり】 この約束だけ受け取り、中身を後から差し込んで組み立てる仕組みが、次の「DI コンテナ経由」です。

8. DI コンテナ経由(DI / 依存性の注入)

ひとことで言うと:必要な部品を自分で作らず、外から用意して渡してもらう仕組みを通すことのこと。

レストランで食材を自分で畑から育てず、仕入れ業者から届けてもらうようなものです。部品(食材)の用意は専門の係(コンテナ)に任せ、調理だけに集中できます。

◆ DI のうれしさ

- 部品の準備を自分で書かなくてよい
- 部品を別のものへ差し替えやすい
- テストのときニセモノ部品に置き換えやすい

【つながり】 差し替えやすさを活かして「同じ窓口で中身だけ切り替える」設計が、次の「ストラテジーパターン」です。

9. ストラテジーパターン(Strategy Pattern)

ひとことで言うと:やり方(戦略)を部品として用意しておき、場面に応じて差し替えられるようにする設計のこと。

料理の「支払い方法」を、現金・カード・電子マネーから選べるレジのようなものです。会計という流れは同じまま、中の方法だけを状況で切り替えられます。

◆ ストラテジーの例(価格計算)

- 通常価格の計算
- セール価格の計算
- 会員割引の計算

【つながり】 こうした処理が扱う「注文」「在庫」といった現実の物事を表す設計が、次の「ドメインモデル」です。

10. ドメインモデル(Domain Model)

ひとことで言うと:現実の業務に出てくる物事(注文・商品・在庫など)を、そのままコードの形に写したもののこと。

お店の現実を写し取った「登場人物カード」のようなものです。「注文」「商品」「在庫」といった現実の概念を、そのまま扱える部品にしておきます。

◆ 注文系のドメインモデル例

- Order(注文): 誰が・何を・いくつ買ったか
- OrderItem(注文明細): 注文に含まれる 1 品ずつ
- Inventory(在庫): 商品が今いくつあるか

【つながり】 こうした本筋の物事とは別に、便利な小道具をまとめておく置き場が、次の「ユーティリティ」です。

11. ユーティリティ(Utility)

ひとことで言うと:どこからでも使える、ちょっとした便利機能を集めた共通の道具箱のこと。

キッチンの引き出しに入った、栓抜きや計量スプーンのようなものです。料理の主役ではないけれど、あちこちで何度も使う小道具をまとめて置いておきます。

◆ ユーティリティの例

- 日付を決まった形に整える
- 文字が空っぽかどうかを判定する
- 金額に税を足して丸める

【つながり】 こうした部品で組み上げられた、実在するクラスの代表例が、次の「SecurityConfig」です。

第3部 — コードの中の実物クラス

12. SecurityConfig(セキュリティ コンフィグ)

ひとことで言うと:「誰がどこまで入ってよいか」という安全のルールをまとめて決める設定クラスのこと。

ビルの入館ルールを決める受付規程のようなものです。どの扉(画面)が誰でも入れて、どの扉は社員証(ログイン)が要るのかを、ここで一括して決めます。

◆ SecurityConfig で決めること

- ログインなしで見たい画面はどれか
- ログインが必要な画面はどれか
- 認証や権限のチェックをどう行うか

【つながり】 この設定の中で「本人確認の合言葉」を発行・確認する担当が、次の「JwtTokenProvider」です。

13. JwtTokenProvider(ジェイダブリューティー トークン プロバイダー)

ひとことで言うと:ログインした人に「本人の証」を発行し、その正しさを確かめる担当クラスのこと。

遊園地の入場時にもらうリストバンドを、発行して確認する係のようなものです。一度ログインすればリストバンド(トークン)を渡され、以降はそれを見せるだけで本人と分かります。

◆ Provider の役割

- ログイン成功時にトークン(本人の証)を発行する
- 送られてきたトークンが正しいか検証する
- トークンから「誰なのか」を取り出す

【つながり】 そのトークンが指す「誰なのか」の実体(利用者情報)を読み出す担当が、次の「UserDetailsServiceImpl」です。

14. UserDetailsServiceImpl(ユーザーディテールズ サービス インプリ)

ひとことと言うと:利用者の名前や権限などのアカウント情報を、保管場所から探して取り出す担当クラスのこと。

会員証の番号から、その人の登録情報を台帳で引いてくる受付係のようなものです。「この人は誰で、何をしてよい人か」を調べ、認証の判断材料として渡します。

◆ Impl が取り出す情報

- 利用者を特定する名前や ID
- パスワード照合用の情報
- その人が持つ権限(できること)

【つながり】 認証を抜けた利用者が実際に使う業務処理の一例が、次の「InventoryAllocationService」です。

15. InventoryAllocationService(インベントリ アロケーション サービス)

ひとことと言うと:注文に対して在庫を割り当てて「この分は確保した」と決める処理を担当クラスのこと。

人気店で、来た注文に倉庫の在庫を取り分けて押さえる係のようなものです。早い者勝ちで奪い合いにならないよう、注文ごとに在庫をきちんと割り当てます。

◆ 在庫割り当てですること

- 注文された数だけ在庫があるか確かめる
- 確保できたら、その分を取り置く(割り当てる)
- 足りなければ割り当てを断る

【つながり】 割り当てと並んで注文処理に欠かせない、値段を決める専門部品が、次の「PricingEngine」です。

16. PricingEngine(プライシング エンジン)

ひとことと言うと:割引やキャンペーンも踏まえて、注文の最終的な値段を計算する専門部品のこと。

レジで会員割引やクーポンを当てて合計を出す計算係のようなものです。商品ごとの値段に、割引やまとめ買いのルールを重ねて、最終的な支払額を弾き出します。

◆ 価格計算で考えること

- 商品それぞれの基本価格
- 会員割引やセールの適用
- 数量による値引きや送料

【つながり】 これらのクラスが正しく動くかをまとめて確かめる仕組みが、次の「テストスイート」です。

17. テストスイート(Test Suite)

ひとことと言うと:たくさんのテストをひとまとめにして、一度に実行できるようにした集まりのこと。

出荷前の製品をまとめて確かめる「検査ライン一式」のようなものです。一つひとつのテストをばらばらに走らせる代わりに、ボタン一つで全部の検査を通します。

◆ テストスイートのうれしさ

- 全テストを一度にまとめて実行できる
- どこが壊れたかを一覧で把握できる
- 直したあと、壊していないか素早く確認できる

【つながり】 検査をすり抜けて本番で問題が起きてしまった事態が、次の「インシデント」です。

第4部 — トラブル対応の現場

18. インシデント (Incident)

ひとことと言うと:システムで起きた、対応が必要な障害やトラブルの出来事のこと。

お店で突然「レジが動かない」と起きる緊急事態のようなものです。放っておくと利用者に迷惑がかかるので、見つけしだい記録し、すぐ対応を始めます。

◆ インシデントの例

- 注文一覧の画面がなかなか開かない
- エラーで処理が止まってしまう
- 一部の利用者だけ操作できない

【つながり】 起きたインシデントを「誰が担当するか」決めて割り振るのが、次の「アサイン」です。

19. アサイン (Assign)

ひとことと言うと:仕事や担当を「この人をお願いします」と割り当てることのこと。

レストランで「このテーブルは田中さん担当」と受け持ちを決めるようなものです。担当をはっきりさせることで、誰がボールを持っているか分からなくなる事態を防ぎます。

◆ アサインで決まること

- 誰がそのインシデントを調べるか
- いつまでに対応するか
- 困ったとき誰に相談するか

【つながり】 担当になって調べ始めると最初に出会う症状の一つが、次の「タイムアウト」です。

20. タイムアウト (Timeout)

ひとことと言うと:決めた時間内に処理が終わらず、待つのをあきらめて打ち切るこのこと。

電話で相手が出ないとき、一定時間で切る留守電切り替えのようなものです。いつまでも待ち続けると全体が止まるので、「ここまで待って駄目なら中断」と線を引きます。

◆ タイムアウトが起きると

- 画面に「時間切れ」のエラーが出る
- 重い処理や遅い検索が原因のことが多い
- 本当の原因は別にあるサインのことがある

【つながり】 打ち切られたとき「どこで止まったか」を教えてくれる手がかりが、次の「スタックトレース」です。

21. スタックトレース(Stack Trace)

ひとことと言うと:エラーが起きるまでに通ってきた処理の道のりを、順に並べて記録したもののこと。

事故現場まで残された足跡をたどる捜査記録のようなものです。「どの処理から、どの処理を呼んで、どこで転んだか」が順番に書かれているので、原因の場所を絞り込めます。

◆ スタックトレースの読み方(イメージ)

```
java.lang.NullPointerException
  at OrderService.getOrderList(OrderService.java:42)    ← ここで発生
  at OrderController.list(OrderController.java:20)      ← その呼び出し元
  ...(さらに上へ続く)...
```

【つながり】 こうした手がかりを、本番で常時集めて見せてくれる監視の道具が、次の「APM ツール」です。

22. APM ツール(Application Performance Monitoring)

ひとことと言うと:アプリの動きや速さを常に見張って、どこが遅いかを教えてくれる監視の道具のこと。

病院の心電図モニターのようなものです。アプリの「脈拍(処理速度や負荷)」を絶えず計測し、異常があればグラフや警告で知らせてくれます。

◆ APM ツールで分かること

- どの画面・処理が遅いか
- 時間のかかっている内訳(どこで詰まったか)
- エラーがいつ・どれだけ出たか

【つながり】 怪しい場所が見えたら、自分の手で動きを止めて中を覗く作業が、次の「デバッグ」です。

23. デバッグ(Debug)

ひとことと言うと:不具合の原因を探して取り除く、原因究明と修正の作業のこと。

水漏れの原因を、配管を一区間ずつ確かめて突き止める点検作業のようなものです。あてずっぽうで直すのではなく、実際の動きを確かめながら原因を一つずつ潰していきます。

◆ デバッグの進め方

- 問題が起きる状況を再現する
- 処理を途中で止めて中の値を確かめる
- 原因を見つけて直し、再発しないか確かめる

【つながり】 デバッグで処理を「途中で止める」ために仕掛ける目印が、次の「ブレークポイント」です。

第5部 — 原因を突き止める(性能問題)

24. ブレークポイント(Breakpoint)

ひとことと言うと:プログラムを指定した行で一時停止させ、その瞬間の中身を覗くための目印のこと。

ビデオの「ここで一時停止」ボタンのようなものです。気になる場面でピタッと止めて、そのときの登場人物(変数の値)をじっくり確認できます。

◆ ブレークポイントでできること

- 指定した行で処理を止める
- そのときの変数の中身を確認する
- 一歩ずつ処理を進めて流れを追う

【つながり】 止めて覗くだけでなく、遅い検索を記録から探す方法が、次の「スロークエリログ」です。

25. スロークエリログ(Slow Query Log)

ひとことと言うと:データベースへの問い合わせのうち、特に時間がかかったものだけを記録した一覧のこと。

レジの「会計に時間がかかった人だけ」を控えた記録のようなものです。全部ではなく遅いものだけが残るので、性能問題の犯人を効率よく見つけられます。

◆ スロークエリログで分かること

- どの問い合わせが遅かったか
- どれくらい時間がかかったか
- それが何回も繰り返されていないか

【つながり】 そのログに並ぶ「データベースへの問い合わせ」そのものの言葉が、次の「SQL」です。

26. SQL(エスキューエル)

ひとことで言うと:データベースに「こういうデータが欲しい」とお願いするための専用の言葉のこと。

図書館の司書に「この条件の本をください」と頼む注文の言い回しのようなものです。決まった形で書くと、膨大なデータの中から欲しいものだけを正確に取り出せます。

◆ 書き方(イメージ)

```
-- 注文テーブルから、全部の注文を取り出す
SELECT * FROM orders;
-- 条件を付けることもできる
SELECT * FROM orders WHERE user_id = 100;
```

【つながり】 この SQL を、コードから呼び出して注文一覧を取りに行く処理が、次の「OrderService.getOrderList()」です。

27. OrderService.getOrderList()(オーダーサービス ゲットオーダーリスト)

ひとことで言うと:注文の一覧を取ってくる、注文処理担当(Service)のメソッドのこと。

ウェイターに「今日の注文を全部見せて」と頼むようなものです。画面からの依頼を受けて、注文データを倉庫(データベース)から集めて返します。今回の遅さの入口になった処理です。

◆ このメソッドの流れ

```
List<Order> getOrderList() {
    // 1. 注文をまとめて取得する
    // 2. 各注文の明細をそろえる
    // 3. 一覧として返す
}
```

【つながり】 この中で実際にデータベースから全件を取りに行く部分が、次の「OrderRepository.findAll()」です。

28. OrderRepository.findAll()(オーダーリポジトリ ファインドオール)

ひとことと言うと:注文データを保管場所(データベース)から「全部取り出す」メソッドのこと。

倉庫番に「棚の商品を全部出して」と頼むようなものです。注文テーブルの行をまとめて読み出します。件数が多いと、ここだけでも時間がかかります。

◆ findAll() の注意点

- データが多いほど取り出しに時間がかかる
- 本当に全件必要かを見直す余地がある
- 明細を別途取りに行くと回数が増えやすい

【つながり】 各注文の明細を 1 件ずつ取りに行き、回数が膨らむ原因になったのが、次の「OrderItemRepository.findById()」です。

第 6 部 — N+1 問題と解決策

29. OrderItemRepository.findById()(オーダーアイテムリポジトリ ファインドバイオーダーアイディー)

ひとことで言うと:ある 1 件の注文に紐づく明細(中身の品物)を取り出すメソッドのこと。

1 枚の注文票について「この注文の品目だけ出して」と頼むようなものです。1 件ずつなら問題なくとも、注文 100 件のために 100 回呼ぶと、回数がふくれ上がります(これが N+1 問題)。

◆ N+1 問題が起きる流れ

```
注文を全部取る          ... 1 回
↓ 注文ごとに明細を取りに行く
明細を取る × 注文の件数    ... N 回
合計 = 1 + N 回のアクセスで遅くなる
```

【つながり】 この「1+N 回」を一気にまとめて取れるようにする解決策が、次の「@EntityGraph」です。

30. @EntityGraph(エンティティ グラフ)

ひとことで言うと:関連するデータをまとめて一度に取ってくるよう指定する、目印(アノテーション)のこと。

買い物で「本体とセットの付属品も一緒に出して」とまとめ頼みするようなものです。注文と明細を別々に何度も取りに行く代わりに、関連まで含めて 1 回で取得します。

◆ 使い方(イメージ)

```
// 注文を取るとき、明細(items)も一緒に取得する
@EntityGraph(attributePaths = {"items"})
List<Order> findAll();
```

【つながり】 @EntityGraph が裏側で組み立てる「まとめて取る問い合わせ」の正体が、次の「JOIN FETCH」です。

31. JOIN FETCH(ジョイン フェッチ)

ひとことで言うと:関連するテーブルを繋いで、必要なデータを一度の問い合わせでまとめて取る書き方のこと。

注文票とその明細を「ホチキスで留めて一緒に受け取る」ようなものです。注文ごとに明細を取りに戻らず、繋いだ状態で1回でそろえるので、アクセス回数がぐっと減ります。

◆ 書き方(イメージ)

```
-- 注文と、その明細を繋いで一度に取る
SELECT o FROM Order o
JOIN FETCH o.items
```

【つながり】 まとめて取れるようになっても件数が多すぎるとき、必要な分だけ小分けにするのが、次の「ページネーション」です。

32. ページネーション(Pagination)

ひとことで言うと:大量のデータを一度に出さず、1ページずつ小分けにして取り出す仕組みのこと。

分厚い本を、ページごとにめくって読むようなものです。検索結果を一気に全部出すと重いので、「1ページ20件ずつ」のように区切って必要な分だけ表示します。

◆ ページネーションのうれしさ

- 1回に扱う件数が減って表示が速くなる
- サーバーやデータベースの負担が軽くなる
- 利用者も一覧を見やすくなる

【つながり】 こうした取り出しの工夫は、注文だけでなく在庫一覧でも使う、次の「InventoryService.getStockList()」にも当てはまります。

33. InventoryService.getStockList()(インベントリサービス ゲットストックリスト)

ひとことと言うと:在庫の一覧を取ってくる、在庫処理担当(Service)のメソッドのこと。

倉庫係に「今ある在庫の一覧を出して」と頼むようなものです。注文一覧と同じく、まとめ取りやページ分けの工夫をしないと、件数が増えたとき遅くなりやすい処理です。

◆ 在庫一覧でも効く工夫

- 関連データはまとめて取る(N+1 を避ける)
- 必要な分だけページで区切る
- 遅い問い合わせはログで見張る

【つながり】 こうした読解と改善の積み重ねが、巨大なコードを「使いこなす」力になり、最初の Wiki 整備へと還っていきます。

付録 A — 全体まとめ表

用語	ひとことと言うと	身近な例え
Wiki	みんなで書く社内の知識ノート	店のバックヤードの業務マニュアル
IntelliJ	高機能な開発専用エディタ	何でも揃ったプロの作業デスク
grep	文字を含む行を一瞬で探す	辞書の索引で単語を引く
エントリーポイント	プログラムの最初の入り口	ビルの正面玄関
コンテキスト	今のコードが動く前提や背景	会話の前後の流れ
レイヤードアーキテクチャ	役割ごとに層で分ける設計	客席・厨房・倉庫の役割分け
インターフェース	できることだけ決める約束	リモコンのボタン配置
DI コンテナ経由	部品を外から渡してもらう	食材を仕入れ業者から届ける
ストラテジーパターン	やり方を差し替えられる設計	支払い方法を選べるレジ
ドメインモデル	現実の物事をコードに写す	現実を写した登場人物カード
ユーティリティ	便利機能を集めた工具箱	引き出しの栓抜きや計量スプーン
SecurityConfig	安全のルールを決める設定	ビルの入館ルール規程
JwtTokenProvider	本人の証を発行・確認する	入場リストバンドの発行係
UserDetailsServiceImpl	利用者情報を探して取り出す	会員番号で台帳を引く受付
InventoryAllocationService	注文に在庫を割り当てる	注文に倉庫の在庫を取り置く係
PricingEngine	割引込みで最終価格を計算	クーポンを当てる計算係

テストスイート	テストをまとめて一度に実行	出荷前の検査ライン一式
インシデント	対応が必要な障害の出来事	レジが動かない緊急事態
アサイン	担当を割り当てる	テーブルの受け持ちを決める
タイムアウト	時間切れで処理を打ち切る	電話の留守電切り替え
スタックトレース	エラーまでの道のりの記録	現場まで残された足跡の捜査記録
APM ツール	アプリの速さを常に監視	病院の心電図モニター
デバッグ	原因を探して取り除く作業	配管を区間ごとに点検
ブレークポイント	指定行で処理を止める目印	ビデオの一時停止ボタン
スロークエリログ	遅い問い合わせだけの記録	会計が遅い人だけの控え
SQL	データベースへ頼む専用言語	司書に本を頼む注文の言い回し
OrderService.getOrderList()	注文一覧を取るメソッド	全注文を見せてと頼む
OrderRepository.findAll()	注文を全部取り出す	棚の商品を全部出して
OrderItemRepository.findById()	1 注文の明細を取る	この注文の品目だけ出して
@EntityGraph	関連をまとめて一度に取る指定	本体と付属品をまとめて頼み
JOIN FETCH	繋いで一度にまとめて取る	注文票と明細をホチキス留め
ページネーション	1 ページずつ小分けに取る	分厚い本をページごとに読む
InventoryService.getStockList()	在庫一覧を取るメソッド	在庫の一覧を出してと頼む

おわりに — 次の一歩

知らない言葉も、身近な例えとセットで覚えると、ぐっと頭に入ります。分からなくなったら、何度でも読み返してください。

◆ こんな進め方がおすすめ

- 「ひとことで言うと」だけ先に通して読む
- 身近な例えとセットで思い出す
- つながりをつたって、関係する用語をまとめて覚える

—— あなたのペースで大丈夫。応援しています! ——